



Vue.js



Firebase



GitHub

PORTFOLIO BASED INTERVIEW STRATEGY

Vue.js 포트폴리오 기반 신입 개발자 기술 면접 가이드

"Vue + Firebase + GitHub 포트폴리오로 말하는 법"



대상

Vue.js 프로젝트 실습을 마치고
취업을 준비하는 **신입 개발자**



목표

단순 구현 설명이 아닌,
구조와 설계 흐름으로 어필하기

GitHub 관리 - “많이”보다 “잘”



피해야 할 태도

× 의미 없는 커밋 남발

```
> git commit -m "수정", "테스트", "123"
```

× 기능 변화 없는 잔디 심기용 커밋

× 결과만 올리고 **과정이 보이지 않는** 저장소

→ "왜"와 "어떻게"가 빠진 코드 덤프(Dump)



면접관이 보는 포인트

✓ 커밋 메시지에 개발자의 의도가 드러나는가

✓ **기능 단위**로 커밋이 분리되어 있는가

✓ 실패 → 수정 → 개선의 흐름이 보이는가

```
- bug: 로그인 실패 시 무한 로딩  
+ fix: 비동기 처리 예외 로직 추가
```

✓ 좋은 커밋 예시 & 면접 멘트

```
git log --oneline

a1b2c3d feat: 로그인 상태에 따른 라우터 접근 제어 추가

e5f6g7h fix: 새로그침 시 로그인 상태 초기화 문제 해결

i8j9k0l refactor: todo 조회 로직을 composable로 분리
```



💬 면접에서 이렇게 말할 수 있어야 함

"기능 하나를 구현할 때 바로 커밋하지 않고,

기본 동작 → 예외 처리 → 구조 정리 순으로 커밋을 나눴습니다."

“어려운 기술”보다 “기초 설계 + 흐름 설명”



흔한 착각 (Bad Case)

- ✗ 최신 라이브러리 **많이 쓰면** 좋아 보일 거라 생각함
(ex. 필요 없는 상황에서의 Redux, Recoil 등 남발)
- ✗ 고급 기술 용어 나열이 곧 **실력** 이라고 착각
- ✗ 코드를 "복사 붙여넣기" 해서 돌아가기만 하면 된다고 생각

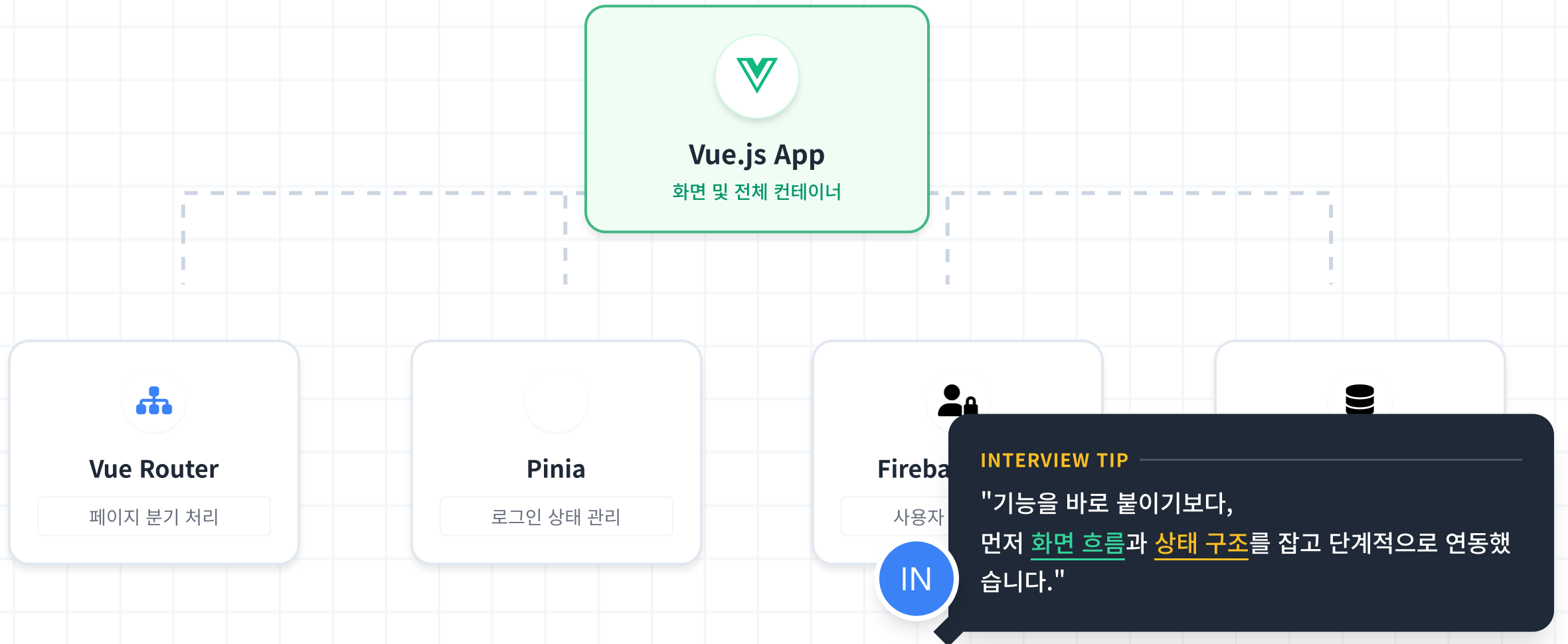


신입 면접 핵심 포인트

- ✓ "왜" 이 구조를 선택했는지 설명할 수 있는가
- ✓ **설계 → 구현 흐름** 을 말로 풀어서 설명할 수 있는가
 - “ 기능을 바로 붙이기보다, 먼저 화면 흐름과 상태 구조를 잡고 단계적으로 연동했습니다.”
- ✓ 기본 개념(LifeCycle, Props, State)을 정확히 이해하고 있는가

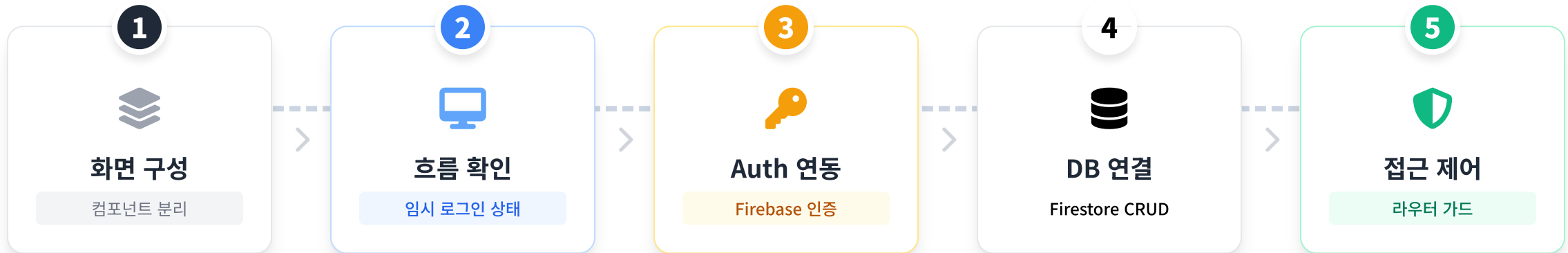
기초 설계 + 흐름 설명 예시

Vue + Firebase Todo 프로젝트의 전체 구조를 한눈에 파악합니다.



구현 순서 Step by Step

기능 단위로 쪼개어 단계적으로 접근하는 것이 핵심입니다.



💬 면접에서 이렇게 설명

"기능을 바로 붙이기보다,
먼저 **화면 흐름** 과 **상태 구조** 를 잡고 단계적으로 연동했습니다."

👉 체계적으로 개발했다는 인상을 심어줄 수 있는 핵심 답변

🧩 사례 1: 새로고침 시 로그인 풀리는 문제



문제 (Problem)

로그인 후 새로고침(F5)을 하면,
인증 정보가 유지되지 않고
다시 로그인 페이지로 튕겨 나감



원인 (Cause)

- ! `auth.currentUser`는
비동기 초기화됨
- ! Firebase가 정보를 가져오기 전에
라우터 가드가 먼저 실행되어
비로그인 상태로 판단함



해결 (Solution)

앱 초기화 시점을 인증 확인 후로 지연시킴

```
onAuthStateChanged(auth, () => {  
  // 인증 상태 확인 후  
  app.mount('#app')  
})
```



면접관에게 어필할 설명 포인트

“ 문제 원인을 막연히 추측하지 않고 **콘솔 로그로 실행 순서를 추적**했으며, Firebase 인증 초기화가 비동기로 동작한다는 **기술적 특성을 이해**하고 해결했습니다. ”

🧩 사례 2: DB엔 없는데 화면에만 추가되는 문제



문제 (Problem)

Todo 리스트에 항목을 추가하면
화면에는 정상적으로 보이지만,
실제 DB(Firestore)에는 데이터가 없음



원인 (Cause)

- ❗ Optimistic UI(낙관적 업데이트)처럼 `todos.push()` 로
화면 갱신을 먼저 수행함
- ❗ Firestore 저장 실패 여부를
확인하는 로직이 누락됨



해결 (Solution)

DB 저장 성공 후에만 상태를 업데이트하고
실패 시 사용자에게 알림

```
try {
  await addDoc(db, todo)
  // 성공 시에만 화면 갱신
  todos.value.push(todo)
} catch (e) { ... }
```



면접관에게 어필할 설명 포인트

“ 단순히 코드를 짰 것이 아니라, 비동기 통신에서 발생할 수 있는 성공/실패 케이스를 분리하지 않았던 설계 실수를 인정하고,
데이터 무결성을 최우선으로 하는 구조로 개선했습니다. ”

🧩 사례 3: 배포 후 새로고침 시 404 오류



문제 (Problem)

Netlify / Cloud Run 배포 후
루트('/')가 아닌 하위 경로
/dashboard 등으로 직접 접근 시
404 Not Found 오류 발생



원인 (Cause)

- ! Vue Router는 브라우저 내부에서만 동작하는 **SPA 라우팅** 방식
- ! 새로고침 시 서버에 해당 경로 파일을 요청하지만,
서버는 `dashboard.html` 파일이 없어서
404를 반환함



해결 (Solution)

모든 경로 요청에 대해 **index.html**을 반환하도록 설정 (SPA Fallback)

```
app.get(/.*/, (req, res) => {  
  res.sendFile(path.join(DIST_DIR,  
    'index.html'))  
})
```



면접관에게 어필할 설명 포인트

“ 단순히 설정 복사/붙여넣기로 끝내지 않고, 클라이언트 사이드 라우팅과 서버 사이드 라우팅의 역할 차이를 명확히 이해하게 된 계기라고 설명했습니다. ”

앞으로 어떻게 확장할 것인가 - "여기서 끝"이 아님을 보여주세요



기능 확장

Feature Extension

✓ Todo 상세화

카테고리 / 우선순위 / 마감일 설정

✓ 사용자별 통계 대시보드

완료율, 주간 달성 그래프

✓ 검색 및 필터링 기능



구조 개선

Refactoring

✓ 로직 분리 (Composable)

`useTodo()`, `useAuth()`

✓ API 레이어 추상화

Firebase 의존성 최소화 설계

✓ 에러 처리 공통화

Toast / Alert 시스템 중앙 관리



운영 관점

DevOps

✓ 환경 변수 분리

`.env.production` / `.env.development`

✓ CI/CD 자동화

GitHub Actions → Netlify 배포

✓ 권한 관리 (RBAC)

관리자 / 일반 사용자 구분



면접관에게 이렇게 말해보세요

"현재는 개인 프로젝트 수준이지만, 사용자 수 증가를 가정해 구조 개선과 운영 자동화를 계획하고 있습니다."

확장 방향을 말로 설계하기

코드를 짜지 않아도 '계획'이 구체적이면 좋은 평가를 받습니다.



현재 상태 인정하기

"현재는 학습 목적의 개인 프로젝트 수준임을 인지하고 있습니다."라고 솔직하게 시작하세요.



사용자 증가 시나리오

트래픽이 늘어날 경우를 가정하여 **DB 구조 개선**이나 **캐싱 도입** 계획을 언급하세요.



측정 가능한 목표 설정

단순히 "빠르게"가 아니라, "응답 시간 0.5초 이내", "오류율 1% 미만" 같은 구체적 지표를 제시하세요.

5

INTERVIEW KEY SPEECH

"현재는 개인 프로젝트 수준이지만,
사용자 수 증가를 가정해
구조 개선을 계획하고 있습니다."



면접관의 반응

"확장성까지 고려할 줄 아는 지원자군" 👍



DB Scaling



API Gateway



Security

“

💡 SECTION 5 : FINAL MESSAGE

면접관이 정말 듣고 싶은 한 문장

“이 프로젝트에서 가장 어려웠던 건
기술이 아니라,
문제를 정의하고 **해결 순서**를 잡는 과정이었습니다.”



면접관의 평가 포인트

이 말이 자연스럽게 나오면 **신입 개발자로서 충분히 준비된 상태**입니다.

”

선택과 배제: 코드를 안 짤 부분

면접관의 숨은 질문: "이 사람은 해야 할 것과 안 해도 될 것을 구분할 줄 아는가?"



실시간 알림 (제외)

"구현 복잡도 대비 포트폴리오에서의 효과가 낮다고 판단하여 제외했습니다."



관리자 페이지 (제외)

"단일 사용자 중심의 프로젝트이므로, 관리자 기능은 범위 밖(Out of Scope)으로 정의했습니다."



백엔드 직접 구현 (대체)

"프론트엔드 상태 관리에 집중하기 위해, 백엔드는 **Firebase**로 대체하여 효율을 높였습니다."

SMART ANSWER

"처음에는 ○○ 기능도 고려했지만,
포트폴리오 목적과 범위를
벗어난다고 판단해 과감히 제외했습니다."

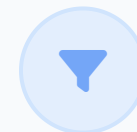


핵심 포인트

구현한 것만큼 '안 한 이유'도 중요합니다.



모든 기능



선택과 집중



완성도 UP

“이 코드가 왜 이 위치에 있는가?”



실제 면접 질문

면접관은 구조적 사고 여부를 확인하고 싶어 합니다.

- ? “이 비즈니스 로직은 왜 **컴포넌트 내부**에 있나요?”
- ? “Pinia를 굳이 사용한 이유가 있나요?
다른 상태 관리 방법은 고려해 보셨나요?”
- ? “혹시 그냥 예제 코드에서 그렇게 해서 따라 한 건 아닌가요?”



좋은 답변의 기준

관심사 분리(Separation of Concerns)를 근거로 설명하세요.

- ✓ UI와 무관한 전역 상태이므로 **Pinia**로 분리했습니다.
- ✓ 여러 컴포넌트에서 재사용될 가능성이 있어 **Component**로 분리했습니다.
- ✓ 역할과 책임을 기준으로 코드 위치를 결정했습니다.

"단순히 예제를 따른 것이 아니라,
유지보수와 재사용성을 고려하여 로직을 분리했습니다."

비동기 처리 제대로 이해하기

async/await을 단순히 '기다리는 문법'이 아니라, '순서 보장'의 관점에서 설명해야 합니다.



흔한 착각 (탈락 포인트)

면접관이 "가장 헛갈렸던 점"을 물었을 때, 단순히 문법 (`async/await`) 사용법만 이야기하면 깊이가 부족해 보입니다.



데이터 무결성 보장

화면 갱신보다 데이터 저장이 완료되는 것이 더 중요했기 때문에 await을 사용하여 순서를 제어했음을 강조하세요.



이벤트 기반 처리

Firebase 인증처럼 값이 즉시 반환되지 않는 경우, `onAuthStateChanged` 같은 **옵저버 패턴**을 활용한 경험을 말하세요.

“

BEST ANSWER EXAMPLE

"화면 갱신보다 데이터 저장이 먼저 보장돼야 해서 await을 통해 순서를 제어했습니다."



면접관의 반응

"비동기 흐름과 실행 컨텍스트를 이해하고 있군" 🙌



Sync Flow



Latency



Stack

에러 처리에 대한 태도

면접관은 '완벽한 코드'보다 '실패를 고려한 코드'를 더 신뢰합니다.



단순 await 금지

`await addDoc(...)` 한 줄로 끝내지 마세요. 네트워크 요청은 언제든지 실패할 수 있습니다.



Try / Catch 의도적 사용

예외 발생 시 앱이 멈추지 않고, 정해진 흐름(fallback)으로 이어지도록 설계했음을 보여주세요.



메시지 분리 (User vs Dev)

사용자에게는 친절한 안내를, 개발자에게는 상세한 로그를 남기는 이중 처리가 핵심입니다.

INTERVIEW KEY SPEECH

"에러는 숨기지 않고
사용자에게는 안내 메시지,
개발자에게는 로그가
남도록 설계했습니다."



면접관의 반응

"운영 관점까지 생각하고 있군." 👏



Log



Alert



Resolve

성능 최적화는 의식부터

직접 구현하지 않았더라도, '어디가 느려질지' 예측할 수 있어야 합니다.



데이터베이스 비용 증가

"데이터가 수천 개로 늘어날 경우, Firestore 전체 조회 쿼리 비용이 급증할 수 있음을 인지하고 있습니다."



무한 렌더링 가능성

"상태 변경이 뷰를 갱신하고, 그 뷰가 다시 상태를 변경하는 **Re-render Loop**를 주의해야 합니다."



Computed / Watch 남용

"편리하다고 무분별하게 사용하면 불필요한 연산이 발생하므로, 필요한 시점에만 동작하도록 설계해야 합니다."

“

INTERVIEW KEY POINT

"신입에게 최적화는 필수가 아니라,
문제가 생길 지점을
예측하는 사고가 더 중요합니다."



면접관의 질문 의도

"단순 기능 구현을 넘어 사이드 이펙트를 고려하는가?" 🤔



Speed



Latency



Memory

협업을 “가정”하고 “회고”하라



협업 가능성 어필

"다른 사람이 이어서 한다면?"



README.md 충실도

실행 방법, 배포 절차, 사용된 기술 스택 명시



폴더 구조와 역할 설명

"왜 이 파일이 여기에 있는지" 설명 가능해야 함



환경 변수(.env) 분리

보안상 중요한 키값 분리 및 주입 방법 안내

“ 혼자 만든 프로젝트라도 항상 다음 사람을 가정하고 문서화했습니다.



마지막 필수 질문

면접관이 묻고 싶은 것

“

"이 프로젝트를
다시 만든다면
무엇부터 바꾸시겠습니까?"



단순 구현을 넘어선 회고 능력 확인



현재 코드의 한계점과 개선안 보유 여부



이 질문에 대한 답이 준비된 사람이 '성장하는 신입'입니다.